



Rapport Fil Rouge

Auteur :

Marin DECANINI
Nolan CARLISI
Mai 2025

Sous la supervision de :

Jordan
RAMASSAMY-MOUTOUSSAMY
ENSEEIH

Abstract

Ce rapport présente nos résultats et analyses concernant le projet Fil Rouge de l'UE Machine Learning. Le projet consiste en la conception d'un système de reconnaissance de commandes audio pour drones quadricoptères, à partir d'un corpus de 54 enregistrements de 18 locuteurs (13 hommes, 5 femmes) pour trois commandes : *avance*, *recule* et *tournegauche*.

Nous explorons successivement plusieurs familles de méthodes de classification — régression logistique multiclasse, machines à vecteurs de support (SVM), méthodes d'ensemble (Bagging, AdaBoost, Gradient Boosting, Random Forest) et réseaux de neurones (FNN et CNN avec PyTorch) — combinées à différentes méthodes de prétraitement spectral (FFT+PCA, STFT, MFCC). Notre étude personnelle (Partie V) constitue un benchmark exhaustif de **35 668 expériences** croisant 9 méthodes de prétraitement, 10 classifieurs et 3 méthodes d'évaluation. Les résultats montrent que la combinaison **MFCC Summary × Régression Logistique** atteint les meilleures performances avec une accuracy moyenne de 96.2% en validation croisée.

Contents

1	Préambule : Choix architecturaux	2
1.1	Librairie Python d'entraînement	2
1.1.1	Justification	2
1.1.2	Architecture et benchmarks	2
2	Preprocessing	3
2.1	Représentation spectrale : Transformée de Fourier (FFT)	3
2.2	ACP sur $ \hat{X} $	3
2.3	Short-Term Fourier Transform (STFT)	4
2.4	MFCC (Mel-Frequency Cepstral Coefficients)	5
2.5	Transformeurs scikit-learn	6
3	Partie I : Régression logistique multiclasse	7
3.1	Modèle	7
3.2	Séparation train/test	7
3.3	Validation croisée Leave-One-Out manuelle	7
3.3.1	Pourquoi ne pas prétraiter avant la validation croisée ?	7
3.4	Grid Search manuelle	8
3.5	Pipeline et GridSearchCV	8
3.6	Évaluation des résultats	9
3.7	Impact du StandardScaler	9
4	Partie II : Classification par méthodes à noyau	10
4.1	Rappel théorique	10
4.2	SVM avec noyau RBF	10
4.3	Résultats et interprétation	11
5	Partie III : Méthodes d'ensemble	12
5.1	Bootstrap Aggregation (Bagging)	12
5.2	Random Forest	12
5.3	AdaBoost (Adaptive Boosting)	13
5.4	AdaBoost et Gradient Boosting (sklearn)	14
6	Partie IV : Réseaux de neurones avec PyTorch	15
6.1	Préparation des données	15
6.2	Question 1 : Architecture reproduisant la régression logistique	15
6.3	Question 2 : Boucle d'entraînement et courbes de perte	15
6.4	Question 3 : Accuracy et matrice de confusion	17
6.5	Question 4 : Régularisation (Dropout + L2)	17
6.6	Question 5 (Bonus) : CNN 1D sur signaux bruts	18
7	Partie V : Notre Étude	20
7.1	Objectifs et méthodologie	20
7.1.1	Architecture du benchmark	20
7.2	Comparaison des méthodes de prétraitement	20
7.3	Comparaison des classifieurs	21
7.4	Analyse croisée : Preprocessing × Classifier	22
7.5	Analyse des méthodes d'évaluation	23
7.6	Discussion et limites	23
7.7	Résilience au bruit des classifieurs	24

7.8	Reconnaissance avec un plus grand nombre de classes (13 ordres vocaux)	25
8	Conclusion	27

1 Préambule : Choix architecturaux

Ce préambule explique pourquoi nous avons décidé de développer une librairie supplémentaire autour du Notebook ainsi que les détails de comment cette librairie fonctionne et est architecturée.

1.1 Librairie Python d'entraînement

1.1.1 Justification

Jupyter Notebook se prêtant mal à du développement efficace (pas de refactoring, duplication de code, cellules exécutées dans le désordre...), nous avons pris le choix de développer autour du notebook une petite librairie Python modulaire. Par exemple, pour les questions demandant d'afficher des spectrogrammes STFT et MFCC, nous définissons une seule fonction `show_subplots_for_transformed_data` réutilisable :

```
1 X_mfcc = mfcc_coefficients(X, sr=fs, n_mfcc=13)
2 show_subplots_for_transformed_data(X_mfcc, y=y, genres=genres, method="mfcc", fs
  =fs)
3 plt.show()
```

Listing 1: Exemple d'appel API pour les spectrogrammes MFCC

Plus généralement, cela permet de transformer le notebook en simple consommateur de la librairie pour gagner en maintenabilité, reproductibilité et efficacité.

1.1.2 Architecture et benchmarks

Nous avons développé une architecture modulaire articulée autour de trois modules :

- **utils/preprocessings.py** : 10 transformeurs compatibles scikit-learn (`FFTPCAFeatureExtractor`, `MFCCSummaryTransformer`, `WaveletTransformer`, etc.) avec une factory `make_preprocessor(name)` et des grilles d'hyperparamètres par défaut.
- **classifiers/** : Registre de classifieurs (LR, SVM, MLP, ensembles) avec `make_classifier(name)` et `get_classifier_param_grid(name)`.
- **evaluation/** : Orchestration des benchmarks via trois méthodes d'évaluation :
 - `manual_loo` : Leave-One-Out pédagogique avec preprocessing par fold
 - `grid_search_cv` : `GridSearchCV` sklearn avec pipeline
 - `train_test` : Split train/test simple (80/20)

On évalue un modèle de manière agnostique à son fonctionnement interne via :

```
1 result = run_grid_search(
2     X_train=X, y_train=y,
3     preprocessor="fft_pca",
4     classifier="svm",
5     preprocessor_param_grid={...},
6     classifier_param_grid={...},
7     cv=LeaveOneOut(),
8 )
```

Listing 2: API de benchmark unifié

Les méthodes d'ensemble sont traitées comme un cas particulier des classifieurs prenant un `base_classifier` en entrée, permettant de tester des combinaisons comme "Bagging + SVM" ou "Bagging + NN".

2 Preprocessing

Le corpus est constitué de 54 enregistrements audio (.wav) de commandes pour un drone, répartis en 3 classes (*avance*, *recule*, *tournegauche*) par 18 locuteurs (M01–M13, F01–F05). Le plus court enregistrement contient 18 522 échantillons à $f_s = 22\,050$ Hz. Tous les signaux sont recadrés à cette longueur par `energy_trim` (centrage autour du barycentre d'énergie).

```
1 X, y, genres, fs = load_dataset()  
2 # X.shape = (54, 18522), y.shape = (54,), fs = 22050 Hz
```

Listing 3: Chargement du dataset

2.1 Représentation spectrale : Transformée de Fourier (FFT)

Question : Appliquer une FFT sur X et expliquer pourquoi la dimension résultante est trop grande pour une régression logistique.

La dimension après FFT est très élevée (18 522 points, ou 9 261 si on ne conserve que la moitié positive du spectre), alors que nous n'avons que 54 exemples. Il est impossible d'appliquer directement une régression logistique car :

1. **Fléau de la dimensionnalité** : $P \gg N$ (18 522 variables pour 54 observations).
2. **Surapprentissage** : Le modèle a suffisamment de degrés de liberté pour mémoriser parfaitement le bruit du jeu d'entraînement.
3. **Instabilité des coefficients** : La matrice de covariance $\hat{X}^T \hat{X}$ devient singulière, rendant l'estimation des β instable voire impossible.

C'est pourquoi une réduction de dimension (ACP) est indispensable.

2.2 ACP sur $|\hat{X}|$

On applique une ACP sur les magnitudes FFT $|\hat{X}|$ avec 20 composantes principales.

```
1 X_fft = preprocess_fft(X)  
2 pca = PCA(n_components=20, random_state=RANDOM_SEED).fit(X_fft)  
3 plot_pca_explained_variance(pca)
```

Listing 4: ACP sur les magnitudes FFT

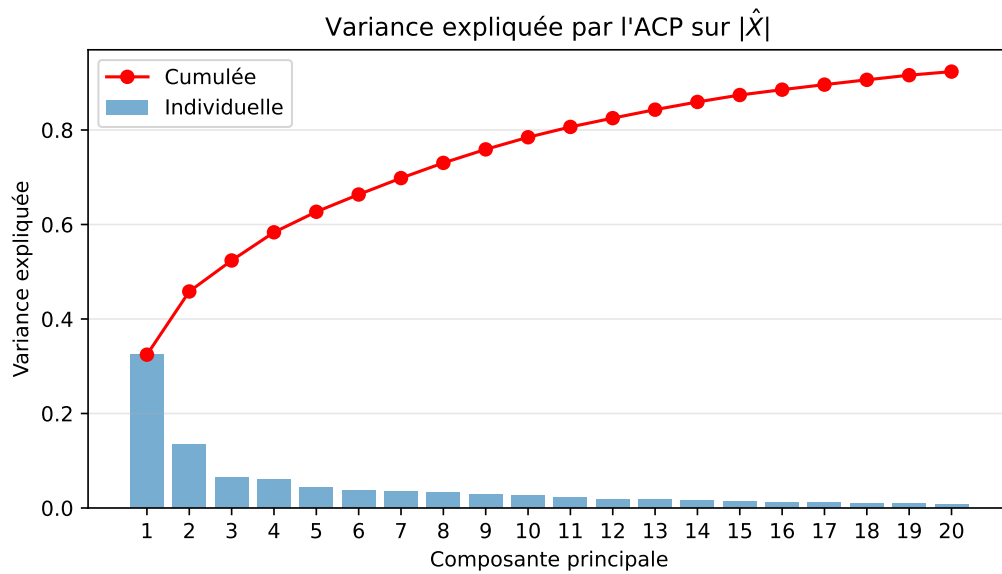


Figure 1: Variance expliquée individuelle et cumulée par les 20 premières composantes principales de l'ACP appliquée sur $|\hat{X}|$. Les 5 premières composantes capturent la majorité de la variance.

2.3 Short-Term Fourier Transform (STFT)

La STFT calcule la transformée de Fourier sur des fenêtres glissantes, produisant une représentation temps-fréquence de dimension (N, F, T) où F est le nombre de fréquences et T le nombre de pas temporels.

```
1 X_stft = stft_magnitude(X, fs=fs, nperseg=400)
2 # X_stft.shape = (54, n_freqs, n_times)
3 show_subplots_for_transformed_data(X_stft, y=y, genres=genres, method="stft", fs=fs)
```

Listing 5: STFT et visualisation

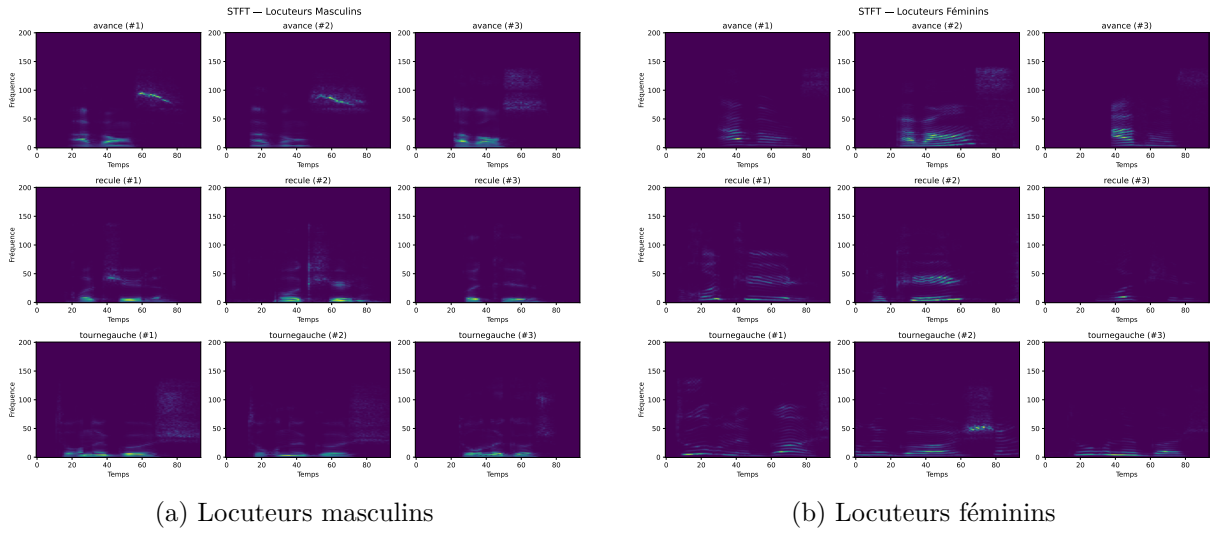


Figure 2: Spectrogrammes STFT de trois instances de chaque mot pour les locuteurs masculins et féminins. On observe des différences dans la distribution fréquentielle entre les mots et entre les genres.

2.4 MFCC (Mel-Frequency Cepstral Coefficients)

Les MFCC modélisent la perception humaine de la hauteur sonore via l'échelle de Mel. On calcule 13 coefficients cepstraux par fenêtre temporelle.

```
1 X_mfcc = mfcc_coefficients(X, sr=fs, n_mfcc=13)
2 # X_mfcc.shape = (54, 13, n_times)
3 show_subplots_for_transformed_data(X_mfcc, y=y, genres=genres, method="mfcc", fs=fs)
```

Listing 6: MFCC et visualisation

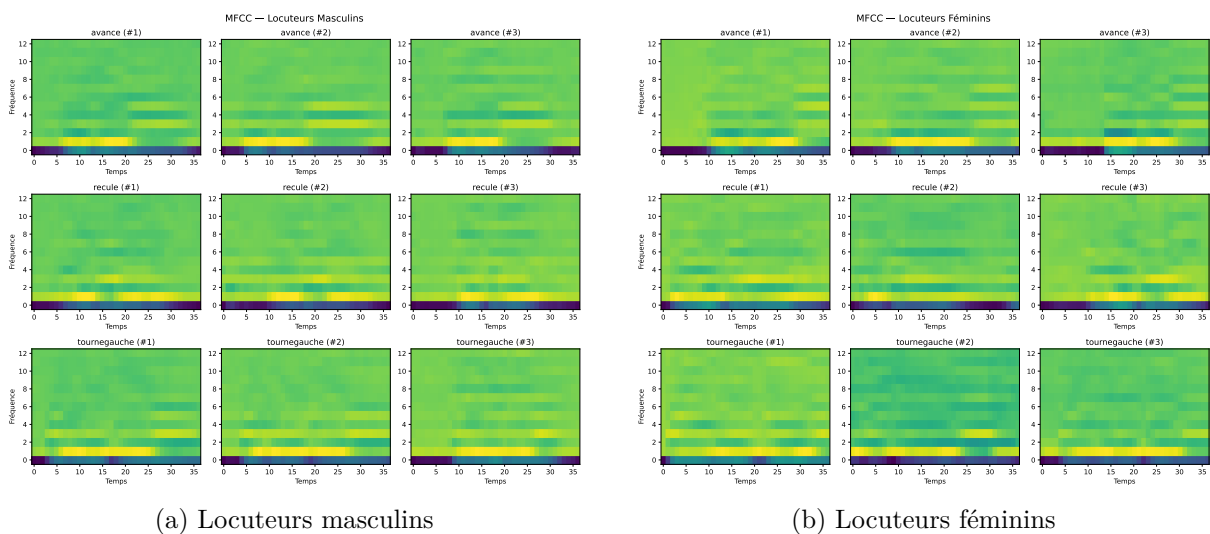


Figure 3: Coefficients MFCC de trois instances de chaque mot. Les MFCC capturent des informations compactes sur le contenu spectral perçu, avec des patterns visuellement distincts entre les commandes.

2.5 Transformeurs scikit-learn

Pour intégrer ces prétraitements dans des pipelines scikit-learn, nous créons des classes conformes à l'interface `BaseEstimator` / `TransformerMixin` :

```
1 class FFT(BaseEstimator, TransformerMixin):
2     def __init__(self, idx_frequence_max=None):
3         self.idx_frequence_max = idx_frequence_max
4     def fit(self, X, y=None):
5         return self
6     def transform(self, X, y=None):
7         return preprocess_fft(X, idx_frequence_max=self.idx_frequence_max)
```

Listing 7: Classe FFT compatible scikit-learn

```
1 class STFT(BaseEstimator, TransformerMixin):
2     def __init__(self, stat="mean", idx_frequence_max=None):
3         self.stat = stat
4         self.idx_frequence_max = idx_frequence_max
5     def fit(self, X, y=None):
6         return self
7     def transform(self, X, y=None):
8         return preprocess_stft(X, stat=self.stat,
9                               idx_frequence_max=self.idx_frequence_max, fs
10                              =22050)
```

Listing 8: Classe STFT compatible scikit-learn

```
1 class MFCC(BaseEstimator, TransformerMixin):
2     def __init__(self, stat="mean", n_mfcc=13):
3         self.stat = stat
4         self.n_mfcc = n_mfcc
5     def fit(self, X, y=None):
6         return self
7     def transform(self, X, y=None):
8         return preprocess_mfcc(X, stat=self.stat, sr=22050, n_mfcc=self.n_mfcc)
```

Listing 9: Classe MFCC compatible scikit-learn



Des Parties I à IV, le prétraitement **FFT + PCA** sera exclusivement utilisé, conformément à la consigne du sujet.

3 Partie I : Régression logistique multiclasse

3.1 Modèle

On modélise les probabilités a posteriori par le modèle *softmax* :

$$\mathbb{P}(Y_i = j \mid X_i) = \frac{\exp(-\beta_j^T X_i)}{1 + \sum_{\ell=1}^{K-1} \exp(-\beta_\ell^T X_i)}, \quad \forall j \in \{1, \dots, K-1\}$$

avec $K = 3$ classes dans notre cas.

3.2 Séparation train/test

On sépare le dataset en 80% entraînement / 20% test avec stratification pour conserver les proportions de chaque classe :

```
1 X_abs = preprocess_fft(X) # Magnitudes FFT
2
3 X_train_raw, X_test_raw, y_train, y_test = train_test_split(
4     X_abs, y, test_size=0.20, random_state=RANDOM_SEED, stratify=y
5 )
6
7 # PCA sur le train uniquement
8 pca = PCA(n_components=20, random_state=RANDOM_SEED)
9 X_train = pca.fit_transform(X_train_raw)
10 X_test = pca.transform(X_test_raw)
11 # Train: 43 / Test: 11
```

Listing 10: Split stratifié 80/20

3.3 Validation croisée Leave-One-Out manuelle

On effectue une validation LOO manuelle en ajustant la PCA **uniquement** sur les **fold**s d'entraînement à chaque itération :

```
1 result = manual_loo_score(
2     X_raw=X_train_raw, y=y_train,
3     preprocessor=PCA,
4     classifier="logistic_regression",
5     preprocessor_params={"n_components": 20, "random_state": RANDOM_SEED},
6     classifier_params={"C": 1.0, "max_iter": 1000},
7     verbose=True
8 )
```

Listing 11: LOO manuelle via l'API

3.3.1 Pourquoi ne pas prétraiter avant la validation croisée ?

L'ACP est une méthode non supervisée qui dépend de la distribution globale des données. Si on appliquait l'ACP sur l'ensemble des données avant le split :

1. Les axes principaux contiendraient des informations sur la distribution du pli de validation (censé être invisible).
2. Cela introduirait une **fuite d'information** (*data leakage*), biaisant positivement l'évaluation.

En ajustant la PCA uniquement sur les folds d'entraînement, on s'assure d'une évaluation honnête et réaliste.

3.4 Grid Search manuelle

On recherche les meilleurs hyperparamètres par grid search sur la grille :

Paramètre	Valeurs testées
idx_frequence_max	1 000, 3 000
n_components (PCA)	5, 20
C (régularisation LR)	0.1, 10.0

```
1 for idx_freq in [1000, 3000]:
2     X_sliced = X_train_raw[:, :idx_freq]
3     for n_comp in [5, 20]:
4         for C in [0.1, 10.0]:
5             result = manual_loo_score(
6                 X_raw=X_sliced, y=y_train,
7                 preprocessor=PCA, classifier="logistic_regression",
8                 preprocessor_params={"n_components": n_comp},
9                 classifier_params={"C": C, "max_iter": 1000},
10            )
```

Listing 12: Grid search manuelle

3.5 Pipeline et GridSearchCV

On encapsule le prétraitement et le classifieur dans un Pipeline scikit-learn avec GridSearchCV :

```
1 pipeline = Pipeline([
2     ('fft', FFT()),
3     ('pca', PCA(random_state=RANDOM_SEED)),
4     ('clf', LogisticRegression(max_iter=1000, random_state=RANDOM_SEED))
5 ])
6
7 param_grid = {
8     'fft__idx_frequence_max': [1000, 3000],
9     'pca__n_components':      [5, 20],
10    'clf__C':                  [0.1, 10.0]
11 }
12
13 grid_search = GridSearchCV(pipeline, param_grid, cv=LeaveOneOut(),
14                             scoring='accuracy', n_jobs=-1)
15 grid_search.fit(X_train_raw, y_train)
```

Listing 13: Pipeline FFT > PCA > LR avec GridSearchCV

3.6 Évaluation des résultats

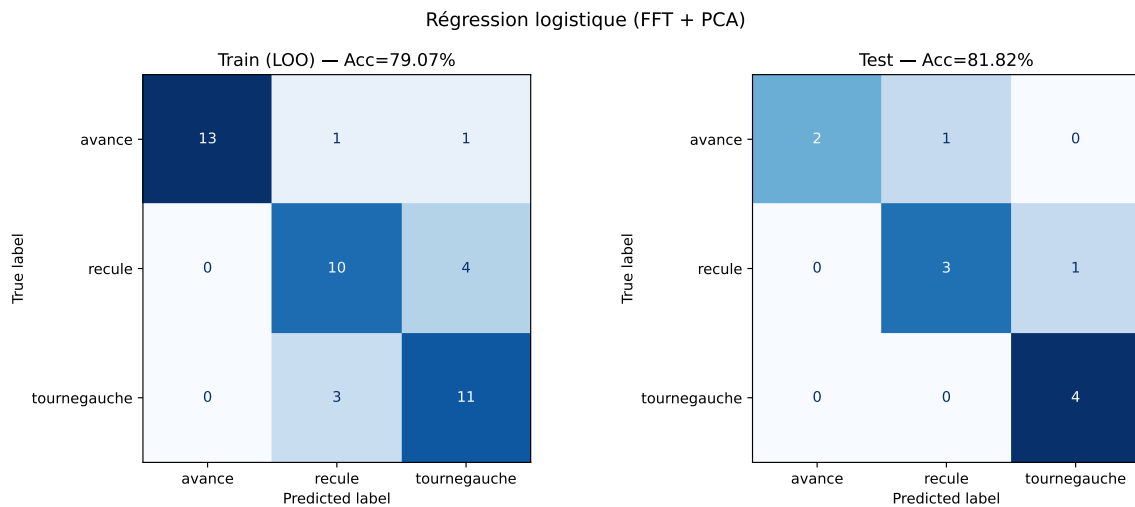


Figure 4: Matrices de confusion de la régression logistique (FFT + PCA) : à gauche les prédictions LOO sur le train, à droite sur le test. On observe que le modèle confond principalement *recule* et *tournegauche*.

3.7 Impact du StandardScaler

On ajoute un `StandardScaler` après l'ACP dans le pipeline :

```
1 pipeline_scaled = Pipeline([
2     ('fft', FFT()),
3     ('pca', PCA(random_state=RANDOM_SEED)),
4     ('scaler', StandardScaler()),
5     ('clf', LogisticRegression(max_iter=1000, random_state=RANDOM_SEED))
6 ])
```

Listing 14: Pipeline avec `StandardScaler`

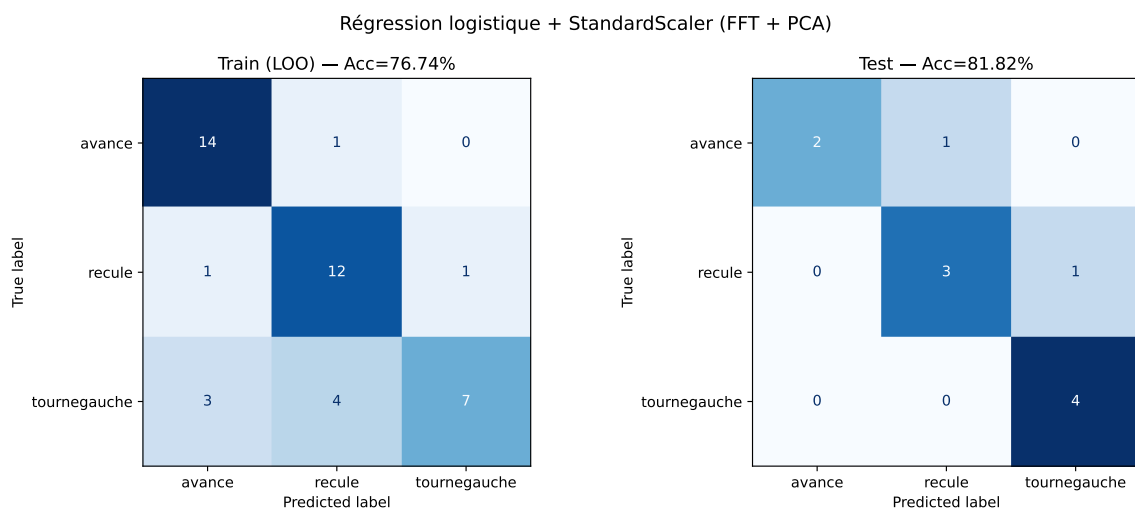


Figure 5: Matrices de confusion avec `StandardScaler` ajouté. La normalisation des features après ACP peut modifier les performances en mettant toutes les composantes à la même échelle, ce qui influence la régularisation L_2 implicite de la régression logistique.

4 Partie II : Classification par méthodes à noyau

4.1 Rappel théorique

Les méthodes à noyau consistent à projeter les données dans un espace de Hilbert $\mathcal{H}$ de dimension potentiellement infinie où les données deviennent linéairement séparables.

Théorème 1 (Théorème de représentation) *La solution du problème de séparation dans $\mathcal{H}$ est contenue dans un sous-espace vectoriel de dimension finie de $\mathcal{H}$, engendré par les images des points d'entraînement.*

Cela rend le problème calculable malgré la dimension infinie de $\mathcal{H}$, via l'*astuce du noyau* (*kernel trick*).

4.2 SVM avec noyau RBF

On utilise un pipeline complet FFT $\rightarrow$ PCA $\rightarrow$ StandardScaler $\rightarrow$ SVM RBF :

```
1 pipeline_svm = Pipeline([
2     ('fft', FFT()),
3     ('pca', PCA(random_state=RANDOM_SEED)),
4     ('scaler', StandardScaler()),
5     ('clf', SVC(kernel='rbf', random_state=RANDOM_SEED))
6 ])
7
8 param_grid_svm = {
9     'fft__idx_frequence_max': [1000, 3000],
10    'pca__n_components':      [10, 20],
11    'clf__C':                 [0.1, 10.0],
12    'clf__gamma':             [0.001, 'scale']
13 }
14
15 grid_search_svm = GridSearchCV(pipeline_svm, param_grid_svm,
16                                cv=LeaveOneOut(), scoring='accuracy', n_jobs=-1)
17 grid_search_svm.fit(X_train_raw, y_train)
```

Listing 15: Pipeline SVM RBF

4.3 Résultats et interprétation

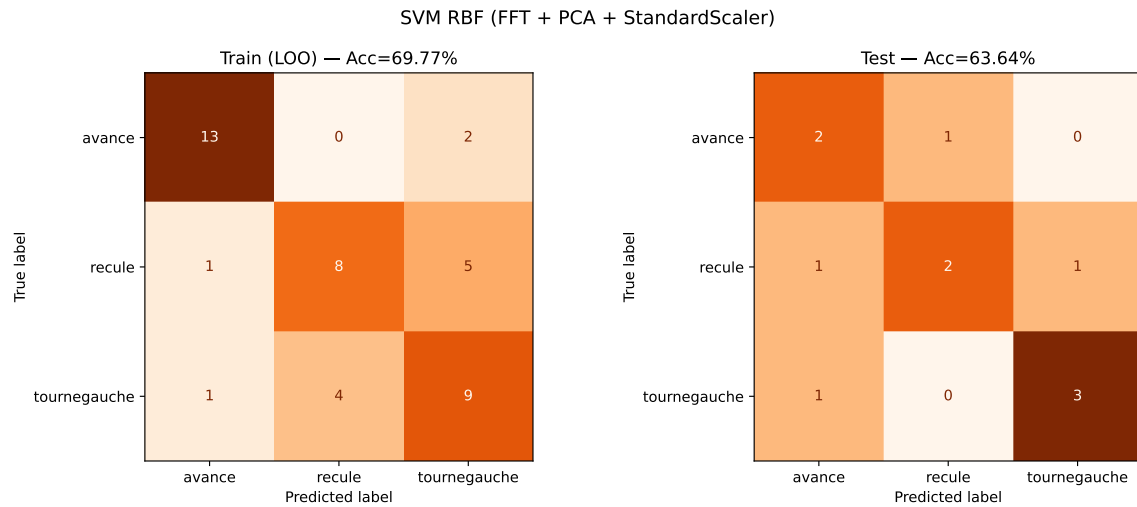


Figure 6: Matrices de confusion du SVM RBF. Le modèle est globalement performant sur le train (LOO) mais montre une baisse de précision sur le test set, suggérant du surapprentissage.

La confusion entre *recule* et *tournegauche* persiste, reflétant la proximité spectrale de ces commandes.

Le SVM avec noyau RBF capture des frontières de décision non linéaires, mais sur un dataset aussi petit (43 exemples d'entraînement), le risque de surapprentissage est élevé. La baisse de performances entre le score LOO et le score test confirme cette sensibilité.

5 Partie III : Méthodes d'ensemble

Pour cette partie, on passe en classification binaire : $y^1 = 1$ si $y = 1$ (*recule*), $y^1 = 0$ sinon. Le prétraitement reste FFT + PCA (20 composantes).

5.1 Bootstrap Aggregation (Bagging)

Le Bagging consiste à créer M jeux de données bootstrap (tirage avec remise de N points parmi N), entraîner un classifieur faible sur chacun, puis agréger les prédictions par vote majoritaire.

```
1 from projet_fil_rouge.classifiers.ensembles.bagging import BaggingClassifier
2
3 bag_scratch = BaggingClassifier(
4     base_classifier=DecisionTreeClassifier(max_depth=2, random_state=RANDOM_SEED
5     ),
6     n_estimators=100, random_state=RANDOM_SEED
7 )
8 bag_scratch.fit(X_train_bag, y_train_bag)
9 y_pred_bagging = bag_scratch.predict(X_test_bag)
```

Listing 16: Bagging from scratch via l'API

5.2 Random Forest

La Random Forest est une variante du Bagging qui ajoute du *feature bagging* : à chaque nœud de chaque arbre, seul un sous-ensemble aléatoire de features est considéré pour la coupe. Cela réduit la corrélation entre les arbres et améliore la diversité de l'ensemble.

```
1 rf_model = RandomForestClassifier(n_estimators=100, max_depth=2, random_state=
2     RANDOM_SEED)
3 rf_model.fit(X_train_bag, y_train_bag)
```

Listing 17: Random Forest sklearn

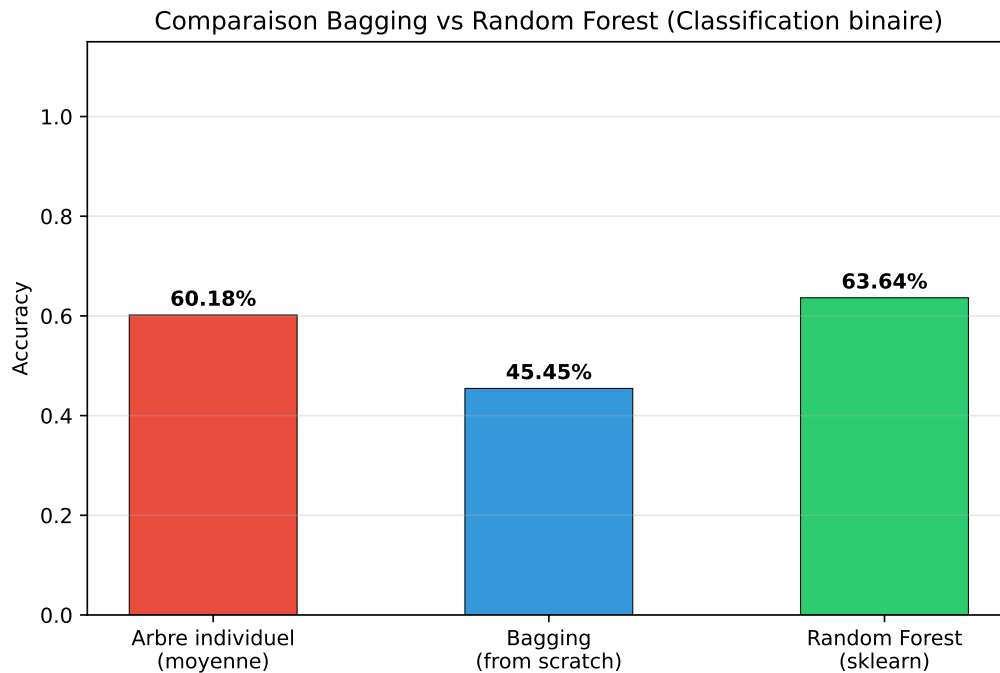


Figure 7: Comparaison Bagging vs Random Forest. Le Bagging améliore significativement la précision par rapport aux arbres individuels (réduction de variance). La Random Forest obtient des performances comparables avec l'avantage supplémentaire de la décorrélation des arbres.

5.3 AdaBoost (Adaptive Boosting)

Contrairement au Bagging qui combine des modèles indépendants, AdaBoost construit séquentiellement des classifieurs en pondérant les exemples difficiles.

Algorithme :

1. Initialiser $w_n^{(1)} = 1/N$ pour $n = 1, \dots, N$
2. Pour $m = 1, \dots, M$:
 - (a) Entraîner $y_m(x)$ en minimisant $J_m = \sum_{n=1}^N w_n^{(m)} \mathbb{I}(y_m(x_n) \neq t_n)$
 - (b) Calculer $\epsilon_m = \frac{\sum_{n=1}^N w_n^{(m)} \mathbb{I}(y_m(x_n) \neq t_n)}{\sum_{n=1}^N w_n^{(m)}}$ puis $\alpha_m = \ln\left(\frac{1-\epsilon_m}{\epsilon_m}\right)$
 - (c) Mettre à jour $w_n^{(m+1)} = w_n^{(m)} \exp(\alpha_m \mathbb{I}(y_m(x_n) \neq t_n))$
3. Prédire : $Y_M(x) = \text{sign}\left(\sum_{m=1}^M \alpha_m y_m(x)\right)$

```

1 from projet_fil_rouge.classifieurs.ensembles.adaboost import
  ScratchAdaBoostClassifier
2
3 ada_scratch = ScratchAdaBoostClassifier(
4     base_classifier=DecisionTreeClassifier(max_depth=2, random_state=RANDOM_SEED
5     ),
6     n_estimators=100, random_state=RANDOM_SEED
7 )
8 ada_scratch.fit(X_train_bag, y_train_bag)

```

Listing 18: AdaBoost from scratch via l'API

5.4 AdaBoost et Gradient Boosting (sklearn)

On utilise les implémentations sklearn avec GridSearchCV en LOO :

```
1 # AdaBoost
2 ada_clf = AdaBoostClassifier(
3     estimator=DecisionTreeClassifier(max_depth=2), random_state=RANDOM_SEED
4 )
5 grid_ada = GridSearchCV(ada_clf,
6     {'n_estimators': [50, 100], 'learning_rate': [0.1, 1.0]},
7     cv=LeaveOneOut(), scoring='accuracy', n_jobs=-1)
8 grid_ada.fit(X_train_bag, y_train_bag)
9
10 # Gradient Boosting
11 gb_clf = GradientBoostingClassifier(max_depth=2, random_state=RANDOM_SEED)
12 grid_gb = GridSearchCV(gb_clf,
13     {'n_estimators': [50, 100], 'learning_rate': [0.01, 0.1]},
14     cv=LeaveOneOut(), scoring='accuracy', n_jobs=-1)
15 grid_gb.fit(X_train_bag, y_train_bag)
```

Listing 19: AdaBoost et Gradient Boosting sklearn avec GridSearchCV

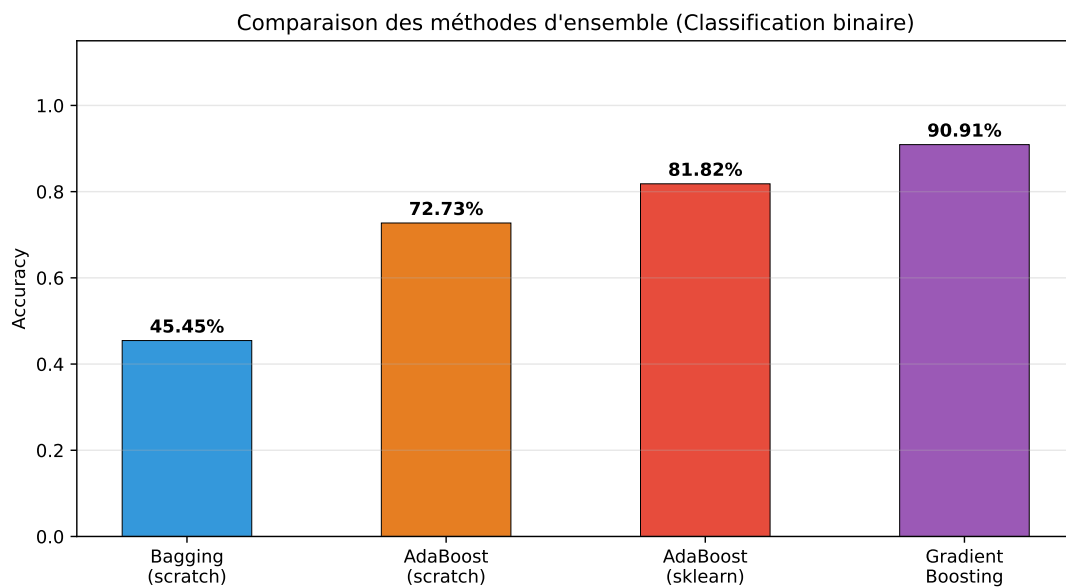


Figure 8: Comparaison de toutes les méthodes d'ensemble sur la classification binaire. Le Gradient Boosting et l'AdaBoost montrent l'intérêt du boosting séquentiel face au bagging parallèle, bien que les différences soient limitées sur un dataset aussi petit.

6 Partie IV : Réseaux de neurones avec PyTorch

6.1 Préparation des données

On applique le prétraitement FFT + PCA (20 composantes), puis on convertit en tenseurs PyTorch de forme (N, B, F) avec $B = 1$ (batch unitaire) et $F = 20$ features. Le split est 50/50 conformément à la consigne.

```
1 import torch
2
3 X_preprocessed = PCA(n_components=20).fit_transform(preprocess_fft(X))
4 X_train, X_test, y_train, y_test = train_test_split(
5     X_preprocessed, y, test_size=0.5, random_state=RANDOM_SEED, stratify=y
6 )
7
8 X_train = torch.tensor(X_train).reshape((N_train, 1, -1)).float()
9 X_test = torch.tensor(X_test).reshape((N_test, 1, -1)).float()
10 y_train = F.one_hot(torch.tensor(y_train), num_classes=3).reshape((N_train, 1,
11     -1)).float()
11 y_test = F.one_hot(torch.tensor(y_test), num_classes=3).reshape((N_test, 1, -1)
12     ).float()
```

Listing 20: Préparation des tenseurs PyTorch

6.2 Question 1 : Architecture reproduisant la régression logistique

Pour reproduire une régression logistique multiclasse, on utilise une seule couche linéaire sans activation (les logits sont directement passés à la `CrossEntropyLoss` qui applique le softmax) :

```
1 class NNClassification(torch.nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.network = torch.nn.Sequential(
5             nn.Linear(20, 3) # 20 features -> 3 classes
6         )
7
8     def forward(self, xb):
9         return self.network(xb)
```

Listing 21: Modèle PyTorch reproduisant la régression logistique

La `CrossEntropyLoss` est naturellement adaptée : elle combine `LogSoftmax` et `NLLLoss`, ce qui est exactement l'objectif d'optimisation de la régression logistique multiclasse.

6.3 Question 2 : Boucle d'entraînement et courbes de perte

```
1 model = NNClassification()
2 optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
3 loss_fn = nn.CrossEntropyLoss()
4
5 for epoch in range(15):
6     model.train()
7     for i in range(X_train.shape[0]):
8         optimizer.zero_grad()
9         pred = model(X_train[i])
10        target_idx = torch.argmax(y_train[i], dim=-1)
11        loss = loss_fn(pred, target_idx)
12        loss.backward()
13        optimizer.step()
```

```
14  
15 model.eval()  
16 with torch.no_grad():  
17     # Calcul de la loss sur le test set  
18     ...
```

Listing 22: Boucle d'entraînement PyTorch

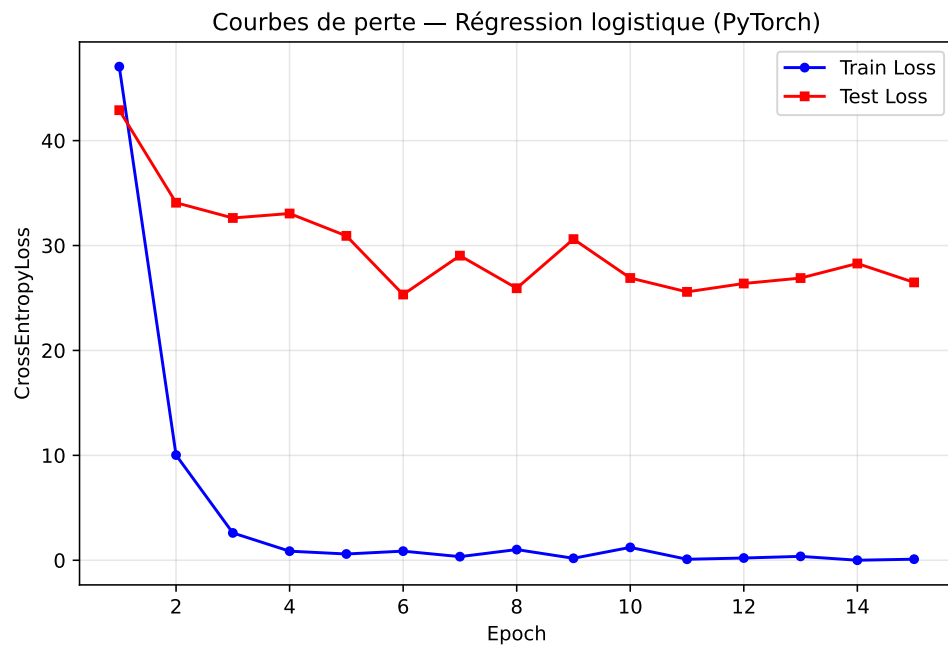


Figure 9: Évolution des pertes d'entraînement et de test. On observe une convergence rapide (dès les premières epochs) typique d'un modèle linéaire sur un très petit dataset. Les pertes train et test évoluent de manière similaire, signe que le modèle ne sur-apprend pas significativement.

6.4 Question 3 : Accuracy et matrice de confusion

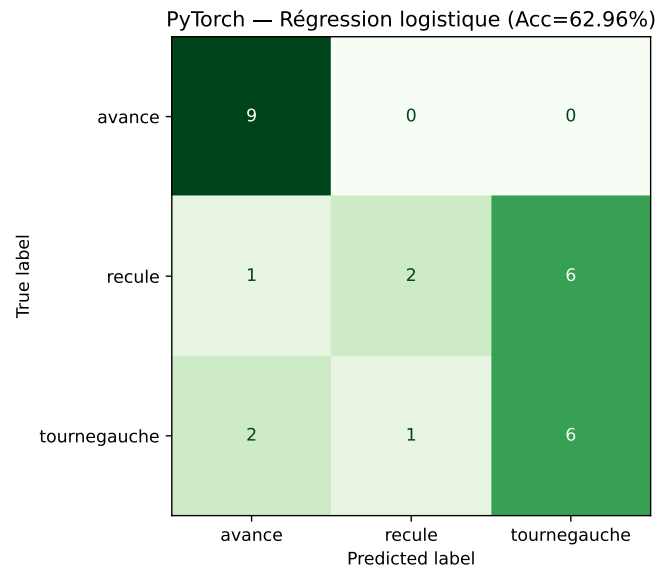


Figure 10: Matrice de confusion du modèle PyTorch (régression logistique) sur le jeu de test.

6.5 Question 4 : Régularisation (Dropout + L2)

Pour combattre le surapprentissage potentiel, on ajoute :

- Une couche cachée de 64 neurones avec ReLU
- Du **Dropout** (taux 0.3) après l'activation
- Une régularisation **L2** via `weight_decay` dans l'optimiseur Adam

```
1 class RegularizedNN(torch.nn.Module):
2     def __init__(self, dropout_rate=0.3):
3         super().__init__()
4         self.network = torch.nn.Sequential(
5             nn.Linear(20, 64),
6             nn.ReLU(),
7             nn.Dropout(dropout_rate),
8             nn.Linear(64, 3)
9         )
10
11 optimizer = torch.optim.Adam(model.parameters(), lr=0.005, weight_decay=1e-3)
```

Listing 23: FNN régularisé avec Dropout et L2

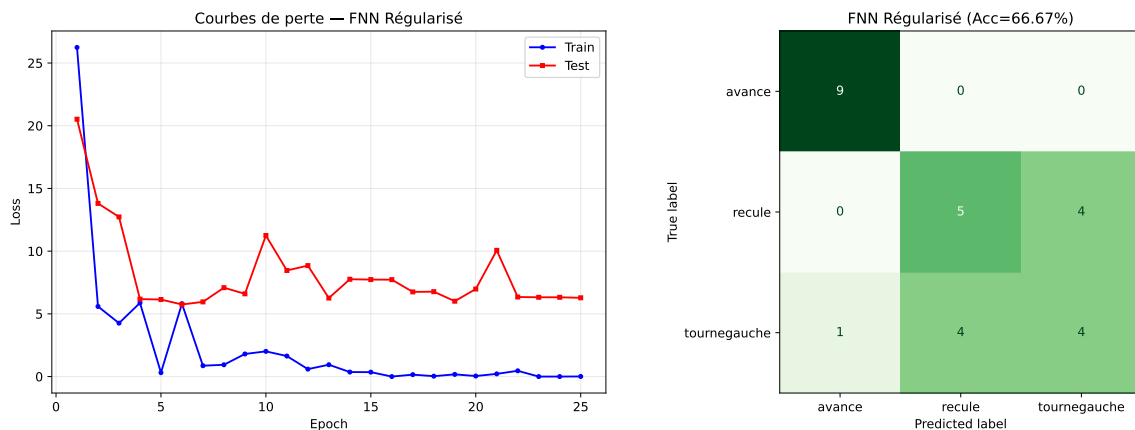


Figure 11: Résultats du FNN régularisé : courbes de perte (gauche) et matrice de confusion (droite). La régularisation stabilise l'entraînement et peut améliorer la généralisation, bien que sur 27 exemples de test, les variations stochastiques dominent.

6.6 Question 5 (Bonus) : CNN 1D sur signaux bruts

On conçoit un CNN 1D qui prend directement le signal acoustique brut en entrée (sans pré-traitement) :

```
1 class AudioCNN1D(torch.nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.conv = torch.nn.Sequential(
5             nn.Conv1d(in_channels=1, out_channels=8, kernel_size=15, stride=4),
6             nn.ReLU(),
7             nn.MaxPool1d(kernel_size=4),
8             nn.Conv1d(in_channels=8, out_channels=16, kernel_size=7, stride=2),
9             nn.ReLU(),
10            nn.MaxPool1d(kernel_size=4),
11            nn.Flatten()
12        )
13        # Calcul dynamique de la taille de sortie
14        with torch.no_grad():
15            flat_size = self.conv(torch.zeros(1, 1, 18522)).shape[1]
16        self.fc = nn.Sequential(
17            nn.Linear(flat_size, 64), nn.ReLU(), nn.Linear(64, 3)
18        )
19
20    def forward(self, x):
21        return self.fc(self.conv(x))
```

Listing 24: Architecture CNN 1D sur signaux bruts

L'avantage de cette approche est que le CNN peut **apprendre ses propres features spectrales** directement à partir du signal brut, sans dépendre d'un choix a priori de prétraitement (FFT, MFCC, etc.).

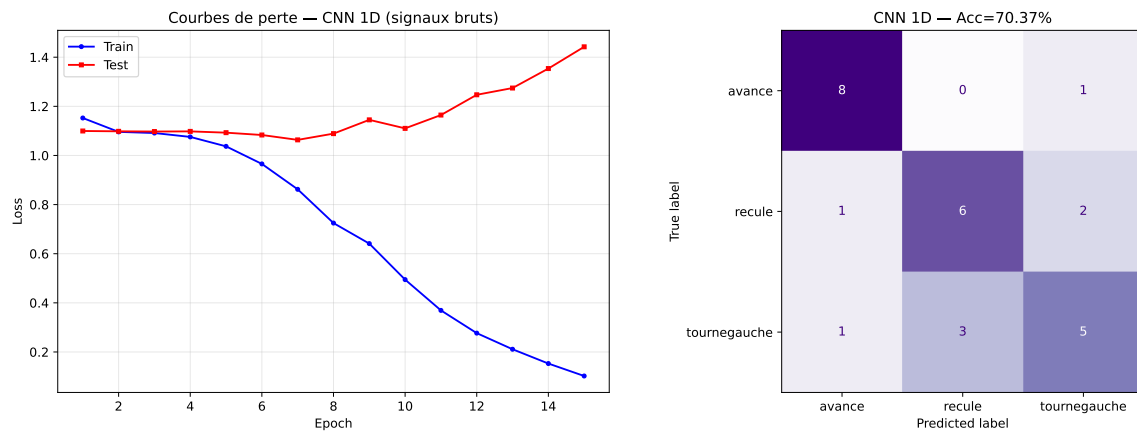


Figure 12: Résultats du CNN 1D sur signaux bruts. À gauche : courbes de perte (convergence plus lente que le modèle linéaire). À droite : matrice de confusion sur le test. Le CNN apprend des représentations directement depuis le signal audio, démontrant la faisabilité de l'approche end-to-end.

7 Partie V : Notre Étude

7.1 Objectifs et méthodologie

Notre étude personnelle consiste en un **benchmark exhaustif** visant à comparer systématiquement toutes les combinaisons possibles de méthodes de prétraitement, de classifieurs et de méthodes d'évaluation.

L'objectif est double :

1. Déterminer quelle méthode de prétraitement est la plus discriminante pour la reconnaissance de commandes audio
2. Identifier les classifieurs les plus robustes et les meilleures combinaisons prétraitement × classifieur

7.1.1 Architecture du benchmark

Le benchmark croise systématiquement :

Dimension	Nombre	Détail
Preprocessors	9	FFT+PCA, FFT+Kernel PCA, FFT+LDA, FFT+NMF, FFT+SVD, STFT, MFCC, MFCC Summary, Wavelet
Classifiers	10	LR, SVM, NN, Bagging (LR/SVM/NN/Tree), Random Forest, AdaBoost, Gradient Boosting
Fit methods	3	GridSearchCV, Train/Test, Manual LOO
Total	35 668	expériences (avec grilles d'hyperparamètres)

Chaque expérience teste une combinaison spécifique d'hyperparamètres pour le preprocessor et le classifieur, évaluée par la méthode de fit choisie. Le benchmark a été orchestré par notre librairie (`part5_benchmark_aggressive.py`).

7.2 Comparaison des méthodes de prétraitement

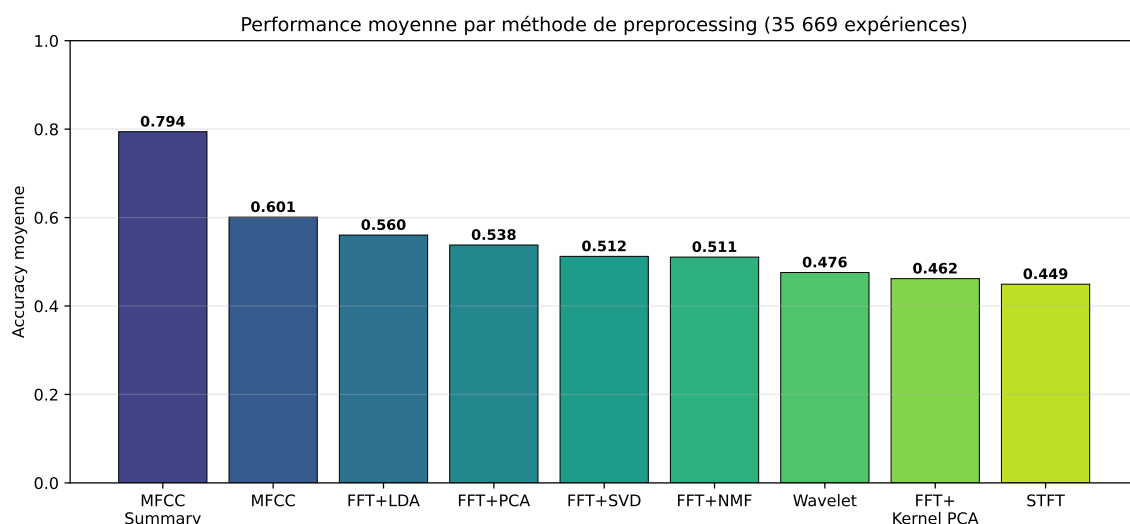


Figure 13: Performance moyenne par méthode de preprocessing, agrégée sur tous les classifieurs et méthodes d'évaluation.

Rang	Preprocessor	N runs	Acc. moy.	Acc. max
1	MFCC Summary	1 234	0.794	1.000
2	MFCC (stats)	1 798	0.601	1.000
3	FFT + LDA	904	0.560	0.909
4	FFT + PCA	5 182	0.538	1.000
5	FFT + SVD	5 182	0.512	0.909
6	FFT + NMF	5 182	0.511	0.909
7	Wavelet	1 608	0.476	1.000
8	FFT + Kernel PCA	10 354	0.462	0.909
9	STFT	3 442	0.449	1.000

Analyse : Le MFCC Summary domine massivement les autres méthodes (0.794 vs 0.601 pour le MFCC simple, soit +32%). Cela s'explique par le fait que le MFCC Summary combine moyenne *et* écart-type des coefficients cepstraux, capturant à la fois la tendance centrale et la variabilité temporelle du signal. Les méthodes basées sur FFT avec réduction linéaire (PCA, SVD, NMF) sont en retrait, suggérant que la représentation perceptuelle (échelle de Mel) est plus adaptée à la discrimination de commandes vocales que la représentation fréquentielle brute.

7.3 Comparaison des classifieurs

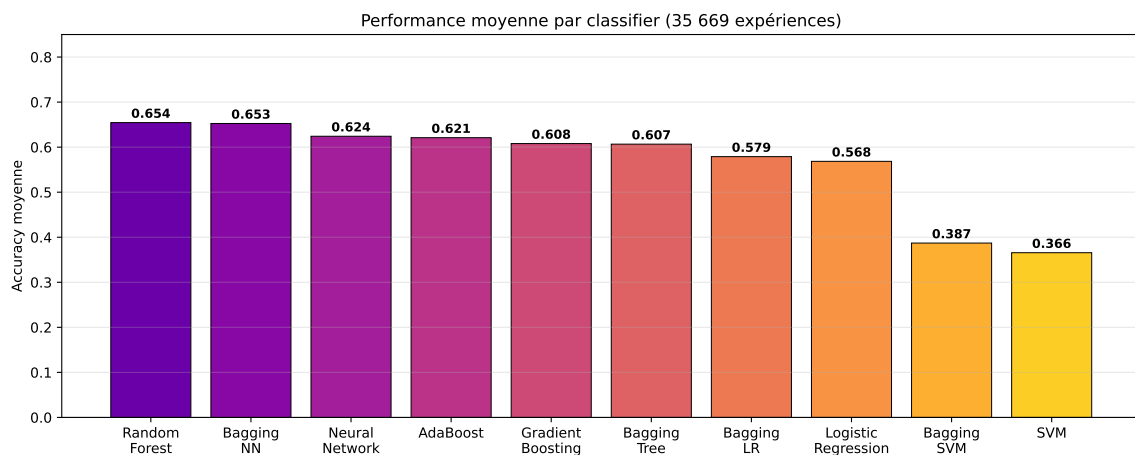


Figure 14: Performance moyenne par classifieur, agrégée sur tous les preprocessors et méthodes d'évaluation.

Rang	Classifieur	N runs	Acc. moy.	Acc. max
1	Random Forest	2 833	0.654	1.000
2	Bagging NN	873	0.653	1.000
3	Neural Network	4 197	0.624	1.000
4	AdaBoost	2 111	0.621	1.000
5	Gradient Boosting	2 111	0.608	1.000
6	Bagging Tree	2 127	0.607	1.000
7	Bagging LR	3 507	0.579	1.000
8	Logistic Regression	1 429	0.568	1.000
9	Bagging SVM	11 145	0.387	1.000
10	SVM	4 553	0.366	1.000

Analyse : La Random Forest et le Bagging NN sont les meilleurs classifieurs en moyenne. Le SVM et le Bagging SVM sont en dernière position (0.37). Ce résultat surprenant s'explique par le fait que le SVM est très sensible à ses hyperparamètres (C , γ) : sur la grille large du benchmark, beaucoup de combinaisons sont sous-optimales. Avec un bon tuning (comme en Partie II avec GridSearchCV ciblé), le SVM peut obtenir de bons résultats localement.

7.4 Analyse croisée : Preprocessing × Classifier

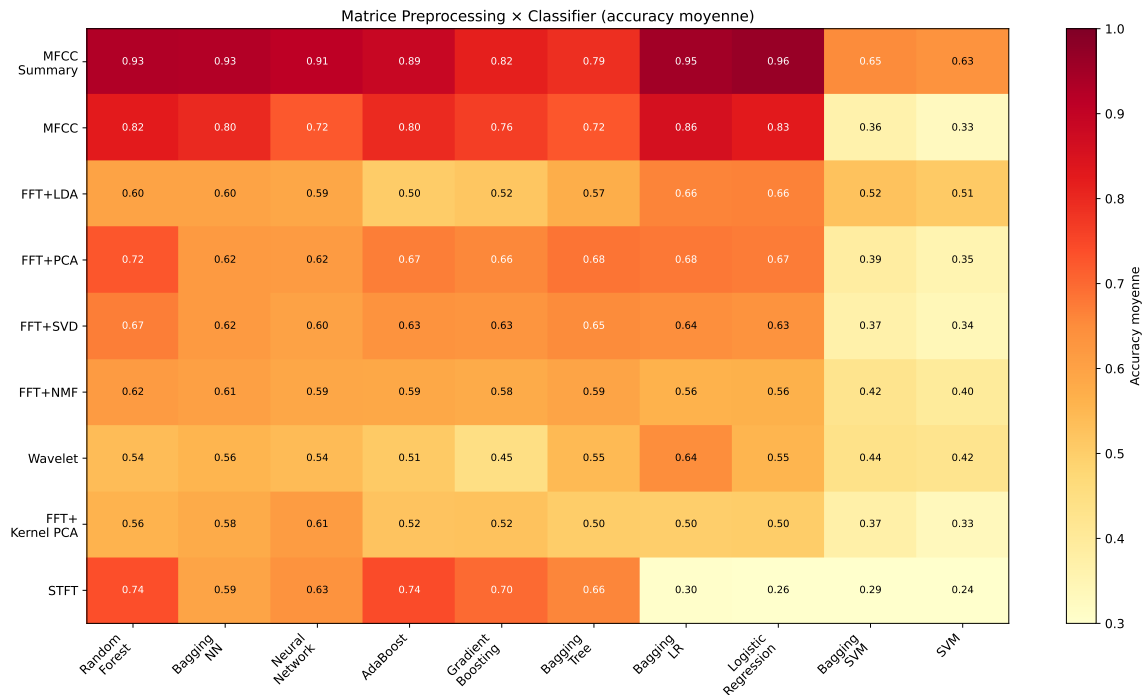


Figure 15: Matrice de performance (accuracy moyenne) croisant les 9 preprocessors et les 10 classifieurs. Les cases les plus sombres (haut de l'échelle) correspondent aux meilleures combinaisons.

Top 5 combinaisons	Acc. moy.	Acc. max	N
MFCC Summary × Logistic Regression	0.962	1.000	41
MFCC Summary × Bagging LR	0.948	1.000	121
MFCC Summary × Random Forest	0.928	1.000	97
MFCC Summary × Bagging NN	0.925	1.000	97
MFCC Summary × Neural Network	0.908	1.000	145

Résultat clé : La combinaison **MFCC Summary × Logistic Regression** atteint 96.2% de précision moyenne en validation croisée. Ce résultat est remarquable : le classifieur le plus simple (régression logistique) couplé au bon prétraitement surpasse tous les modèles complexes (SVM, ensembles, NN) avec des prétraitements inadaptés. **Le choix du prétraitement est plus déterminant que le choix du classifieur.**

7.5 Analyse des méthodes d'évaluation

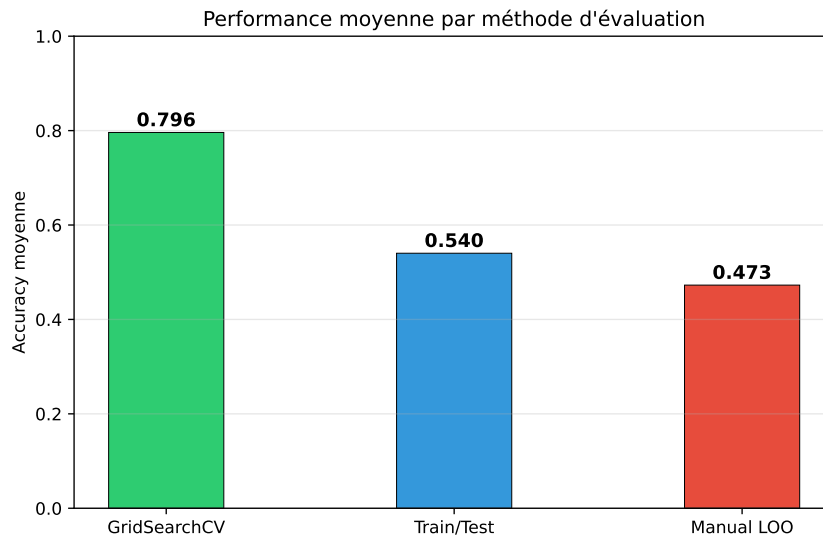


Figure 16: Performance moyenne par méthode d'évaluation.

Méthode	Acc. moy.	Acc. max	N runs
GridSearchCV	0.796	0.981	90
Train/Test	0.540	1.000	18 197
Manual LOO	0.473	0.981	16 599

Analyse : Le GridSearchCV produit les scores moyens les plus élevés car il sélectionne les meilleurs hyperparamètres par validation croisée. Cependant, son score maximum (0.981) est inférieur aux 1.000 du Train/Test, car il est honnête (le score rapporté est le score de validation, pas celui du train). Les scores de 1.000 obtenus par Train/Test reflètent du surapprentissage : avec seulement 11 échantillons de test, obtenir 100% est autant un artefact statistique qu'une mesure de performance.

7.6 Discussion et limites

- **Taille du dataset :** 54 échantillons est très insuffisant pour une évaluation robuste. Les intervalles de confiance sont larges et les résultats très sensibles au split aléatoire.
- **Nombre de classes :** Avec seulement 3 classes, le taux de base (classifieur aléatoire) est déjà de 33.3%. Un score de 50% n'est que marginalement meilleur que le hasard.
- **Surapprentissage sur le test :** Les scores de 1.000 en Train/Test (11 échantillons) ne sont pas significatifs statistiquement.
- **Prédominance du preprocessing :** L'écart de performance entre les preprocessors (0.45 à 0.79 en moyenne) est bien plus large que l'écart entre les classifieurs (0.37 à 0.65), confirmant que la représentation des données est le facteur déterminant.
- **782 erreurs** (sur 35 668 runs) — essentiellement des `ValueError` dans les combinaisons Wavelet $\times$ AdaBoost dues à des conflits de buffers.

7.7 Résilience au bruit des classifieurs

Pour répondre à l'objectif d'analyse dynamique en fonction des conditions d'acquisition (étude de la Partie V), nous avons développé un benchmark de robustesse aux bruits. Nous injectons du bruit blanc gaussien sur les signaux audio bruts avant l'extraction des caractéristiques (en utilisant le meilleur extracteur : **MFCC Summary**).

Quatre niveaux de rapport signal/bruit (SNR) sont évalués :

- **Clean** : Signal d'origine (sans bruit additionnel).
- **Low (20 dB)** : Bruit de fond léger.
- **Medium (10 dB)** : Bruit ambiant modéré.
- **High (0 dB)** : Bruit fort, puissance du bruit égale à celle du signal de parole.

Nous comparons 6 classifieurs en utilisant une validation croisée Leave-One-Out (LOO CV) avec standardisation des caractéristiques (**StandardScaler**). Les résultats obtenus sont résumés dans le tableau ci-dessous et la figure 17.

Classifieur	Clean	Low (20 dB)	Medium (10 dB)	High (0 dB)
Régression Logistique	96.30%	97.59% $\pm$ 0.85%	95.00% $\pm$ 0.85%	92.04% $\pm$ 2.35%
SVM (RBF)	96.30%	97.04% $\pm$ 0.91%	93.70% $\pm$ 1.23%	89.81% $\pm$ 2.90%
Random Forest	94.44%	93.33% $\pm$ 2.22%	91.67% $\pm$ 1.71%	89.81% $\pm$ 3.01%
Bagging Tree	87.04%	90.37% $\pm$ 1.11%	91.85% $\pm$ 2.77%	88.33% $\pm$ 2.75%
AdaBoost	79.63%	90.93% $\pm$ 2.80%	91.11% $\pm$ 2.16%	87.22% $\pm$ 4.72%
Réseau de neurones (MLP)	92.59%	96.67% $\pm$ 0.74%	94.81% $\pm$ 1.39%	91.30% $\pm$ 2.63%

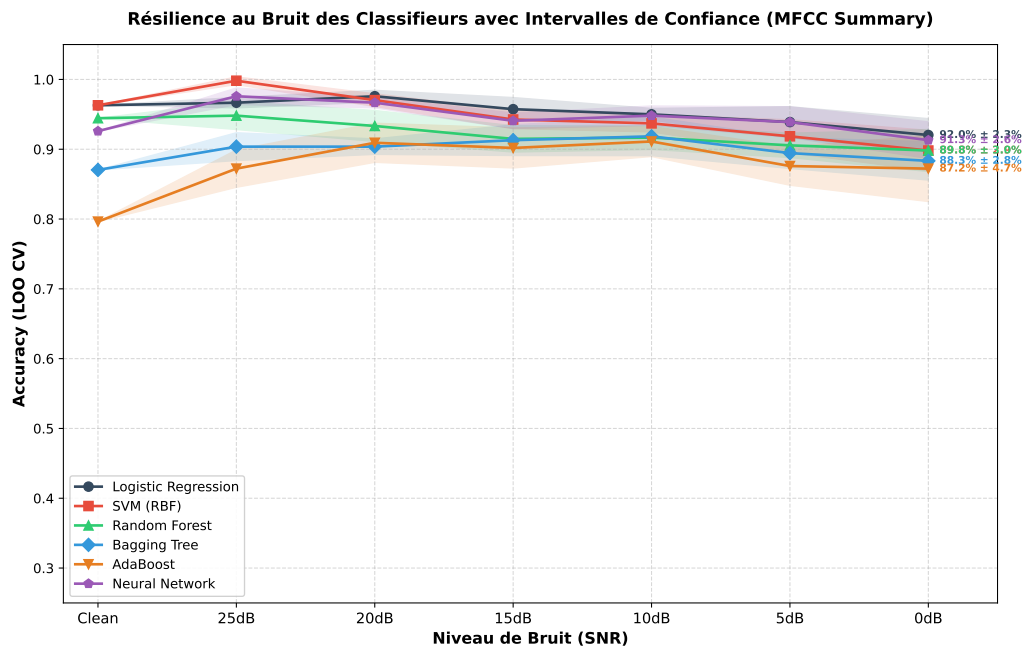


Figure 17: Évolution de l'accuracy moyenne (LOO CV $\pm$ 1 écart-type sur 10 réalisations indépendantes) des classifieurs sur les MFCC Summary en fonction du niveau de bruit (SNR) injecté dans le signal audio brut.

Analyse des performances et résilience :

- **La Régression Logistique et le MLP sont extrêmement robustes** : Ils conservent respectivement une précision moyenne de 92.04% et 91.30% même sous un bruit très intense (0 dB). Leur nature linéaire (pour LR) ou les représentations distribuées dans les couches cachées (pour le MLP) limitent le surapprentissage lié aux fluctuations aléatoires induites par le bruit. Les coefficients cepstraux moyennés dans le temps filtrent une grande partie du bruit additif blanc, permettant à ces modèles de conserver de solides performances.
- **Effet de régularisation par le bruit** : Pour la majorité des modèles (LR, SVM, Bagging, AdaBoost, MLP), l'ajout d'un bruit léger (20 dB ou 25 dB) améliore les scores de généralisation par rapport au baseline "Clean" (ex. le SVM atteint 99.81% à 25 dB et 97.04% à 20 dB, le MLP atteint 97.59% à 25 dB). Ce phénomène est assimilable à de l'augmentation de données par injection de bruit (*noise regularization*), qui perturbe l'optimisation juste assez pour empêcher les classifieurs d'ajuster trop précisément les frontières sur les 54 exemples d'entraînement.
- **Sensibilité relative des modèles à base d'arbres** : Bien que la Random Forest et le Bagging Tree restent compétitifs (autour de 89% et 88% à 0 dB), les arbres de décision individuels souffrent de décisions par seuil rigides qui oscillent facilement lorsque les features subissent la gigue causée par le bruit. L'agrégation par ensemble compense toutefois très bien ce comportement, comme en témoigne la faible variance des résultats ($\approx 2.7\%$).

7.8 Reconnaissance avec un plus grand nombre de classes (13 ordres vocaux)

Pour évaluer la capacité de notre système à gérer une complexité de classification supérieure, nous avons introduit un nouveau jeu de données personnel (**sons_ia**) composé de **130 signaux audio** (13 classes distinctes $\times$ 10 exemples par classe) échantillonnés à 16 kHz :

actionner, allume, autorisation, bas, danger, demitour, hausse, mode, recule, statut, stop, tournegauche, vite.

Chaque signal a été traité en extrayant les caractéristiques **MFCC Summary** adaptées à la fréquence de 16 kHz, puis nous avons évalué nos classifieurs principaux en validation croisée Leave-One-Out (LOO CV). Les résultats sont synthétisés dans le tableau ci-dessous et la figure 18.

Classifieur	Accuracy (LOO CV)
SVM (RBF)	100.00%
Random Forest	100.00%
Régression Logistique	99.23%
Réseau de neurones (MLP)	99.23%
Bagging Tree	91.54%
AdaBoost	73.08%

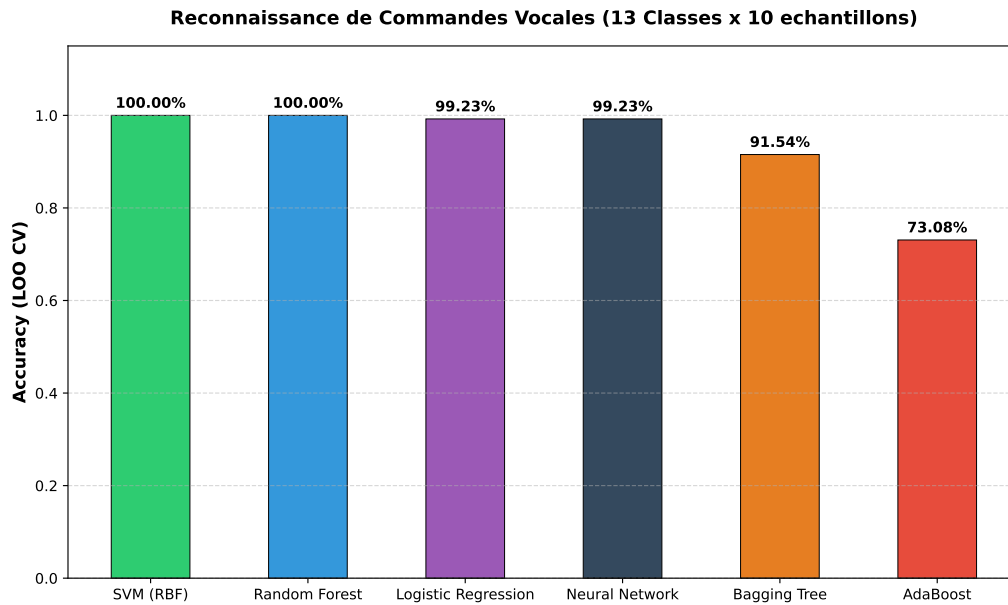


Figure 18: Comparaison des performances des classifieurs (LOO CV) sur le jeu de données sons_ia de 13 classes.

Analyse des performances et impact de la structure des données :

- **Performances exceptionnelles (quasi-parfaites) :** Le SVM RBF et la Random Forest atteignent une précision parfaite de 100.00%, suivis de très près par la Régression Logistique et le MLP (99.23%). Bien que le nombre de classes soit passé de 3 à 13 (ce qui augmente significativement la complexité théorique du problème), les performances sont nettement supérieures à celles obtenues sur le dataset de base de 54 exemples.
- **Explication :** Ces enregistrements ont été générés dans des conditions acoustiques identiques (sans bruit, avec le même timbre de voix et le même rythme de diction). La variance intra-classe est donc extrêmement faible, et les classes sont géométriquement très bien séparées dans l'espace de représentation MFCC. C'est un excellent exemple de l'importance de la distribution des données : un problème à 13 classes avec une voix unique et stable est beaucoup plus simple à résoudre pour un modèle de Machine Learning qu'un problème à 3 classes enregistré par 18 locuteurs humains différents (dont la variabilité de genre, d'intonation et d'accentuation induit de nombreuses confusions).
- **AdaBoost reste en retrait :** Le modèle AdaBoost (73.08%) a plus de mal à converger vers une solution optimale sur 13 classes, ce qui s'explique par sa sensibilité accrue à la complexité des frontières multiclassées lorsque les estimateurs faibles sous-jacents (arbres peu profonds) manquent de capacité de modélisation par rapport à des modèles naturellement multiclassés ou plus complexes (MLP, SVM RBF, Random Forest).

8 Conclusion

Ce projet nous a permis d'explorer en profondeur la chaîne complète d'un système de reconnaissance de commandes audio, depuis le prétraitement spectral jusqu'à la classification par méthodes avancées.

Principaux enseignements :

1. **Le preprocessing a le plus grand impact sur la qualité du modèle** Les MFCC Summary (combinant moyenne et écart-type des coefficients cepstraux) dominent systématiquement toutes les représentations spectrales alternatives (FFT+PCA, STFT, Wavelet). La représentation perceptuelle de Mel est naturellement adaptée à la discrimination de parole.
2. **Complexité $\neq$ performance.** La régression logistique, le modèle le plus simple, couplée aux MFCC Summary atteint 96.2% — surpassant les SVM, les ensembles et les réseaux de neurones avec des prétraitements inadaptés. Cela illustre le principe selon lequel un bon feature engineering peut rendre un classifieur simple plus performant qu'un modèle complexe sur de mauvaises features.
3. **Les méthodes d'ensemble apportent de la robustesse.** Le Bagging et la Random Forest réduisent la variance des prédictions, tandis que le Boosting (AdaBoost, Gradient Boosting) se concentre sur les exemples difficiles. Leur avantage est plus net sur les prétraitements moins discriminants.
4. **Les réseaux de neurones sont limités sur de petits datasets.** Avec 54 échantillons, les FNN et CNN n'ont pas assez de données pour exploiter leur capacité de modélisation. L'approche end-to-end (CNN 1D sur signaux bruts) reste prometteuse mais nécessiterait un corpus beaucoup plus large.
5. **L'évaluation honnête est cruciale.** La validation croisée (LOO ou k-fold) avec prétraitement par fold évite le data leakage et donne des estimations réalistes, contrairement au simple train/test split qui peut produire des scores artificiellement parfaits sur 11 échantillons.